

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 3 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	B V Chaitanya Reddy	Reg. No.:	192425376
Section / Batch:	2024	Date:	

Code of Conduct

I B V Chaitanya Reddy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____chaitanya_____

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Task 1 – N-gram Based Text Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:

"Artificial intelligence is"

the system should identify and display the top-5 most probable next words.

The program should allow the user to select $N = 1, 2$, or 3 , display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.

Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights.

The system should generate the top-5 next-word predictions using:

- Unsmoothed N-gram model
- Backoff model
- Deleted Interpolation model

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:

"The cat sits on the mat."

and

"The quantum processor redesigned the..."

The system should evaluate how predictable the next word is based on the trained language model.

The program should also compare entropy values obtained using:

- Unigram model
- Bigram model
- Trigram model
- Smoothed model

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- Word/tag frequencies
- Tag transition probabilities
- Probabilities obtained from a training corpus

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket."

and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1	3	3	3	4	4	17	81%
2	4	4	3	3	3	17	
3	3	3	3	4	4	17	
4	4	3	3	4	4	18	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. N-gram Based Next-Word Prediction:

Code:

Q1: N-GRAM NEXT WORD PREDICTION

```
from collections import Counter
import re

corpus = """
artificial intelligence is changing the world
artificial intelligence is used in healthcare
artificial intelligence is used in education
machine learning is a branch of artificial intelligence
machine learning can analyze data
machine learning can make predictions
deep learning is a part of machine learning
natural language processing understands language
language models predict the next word
smart text input can suggest the next word
"""

# Tokenization
words = re.findall(r'\b\w+\b', corpus.lower())

# Create N-grams
uni = Counter(words)
bi = Counter(zip(words[:-1], words[1:]))
tri = Counter(zip(words[:-2], words[1:-1], words[2:]))

print("Top Unigrams:", uni.most_common(5))
print("Top Bigrams:", bi.most_common(5))
print("Top Trigrams:", tri.most_common(5))

# Probability functions
def P_uni(w):
    return uni[w] / len(words)

def P_bi(w1, w2):
    return bi[(w1,w2)] / uni[w1] if uni[w1] else 0

def P_tri(w1, w2, w3):
    return tri[(w1,w2,w3)] / bi[(w1,w2)] if bi[(w1,w2)] else 0

# Next-word prediction
def predict(sentence, n=3):
    w = sentence.lower().split()
    result = []

    if n == 1:
        for word in uni:
            result.append((word, P_uni(word)))
```

```

elif n == 2:
    last = w[-1]
    for (a,b) in bi:
        if a == last:
            result.append((b, P_bi(a,b)))

elif n == 3:
    a,b = w[-2],w[-1]
    for (x,y,z) in tri:
        if x == a and y == b:
            result.append((z, P_tri(x,y,z)))

return sorted(result, key=lambda x:x[1], reverse=True)[:5]

```

```
sentence = "artificial intelligence"
```

```

for n in [1,2,3]:
    print("\nN =", n)
    print(predict(sentence,n))

```

```

# Unseen N-gram
print("\nUnseen probability:",
      P_tri("artificial","intelligence","banana"))

if P_tri("artificial","intelligence","banana") == 0:
    print("Unseen N-gram gives zero probability.")

```

Output:

```

... Top Unigrams: [('is', 5), ('learning', 5), ('artificial', 4), ('intelligence', 4), ('machine', 4)]
Top Bigrams: [([('artificial', 'intelligence'), 4], [([('machine', 'learning'), 4], [([('intelligence', 'is'), 3], [([('is', 'used'), 2], [([('used', 'in'), 2]]
Top Trigrams: [([([('artificial', 'intelligence', 'is'), 3], [([('intelligence', 'is', 'used'), 2], [([('is', 'used', 'in'), 2], [([('learning', 'is', 'a'), 2], [([('machine', 'learning', 'is'), 2]]

N = 1
[('is', 0.07936507936507936), ('learning', 0.07936507936507936), ('artificial', 0.06349206349206349), ('intelligence', 0.06349206349206349), ('machine', 0.06349206349206349)]

N = 2
[('is', 0.75), ('machine', 0.25)]

N = 3
[('is', 0.75), ('machine', 0.25)]

```

Q2. Backoff and Deleted Interpolation :

Code:

Q2: BACKOFF AND INTERPOLATION

```
from collections import Counter
import re

corpus = """
machine learning can analyze data
machine learning can make predictions
machine learning is useful
artificial intelligence can solve problems
artificial intelligence is powerful
deep learning can process images
natural language processing can understand text
language models can predict words
data science uses machine learning
"""

words = re.findall(r'\b\w+\b', corpus.lower())

uni = Counter(words)
bi = Counter(zip(words[:-1], words[1:]))
tri = Counter(zip(words[:-2], words[1:-1], words[2:]))

# Probabilities
def p1(w):
    return uni[w] / len(words)

def p2(a,b):
    return bi[(a,b)] / uni[a] if uni[a] else 0

def p3(a,b,c):
    return tri[(a,b,c)] / bi[(a,b)] if bi[(a,b)] else 0

# Backoff: trigram -> bigram -> unigram
def backoff(sentence):

    w = sentence.split()
    a,b = w[-2],w[-1]
    result = []

    # Try trigram
    for (x,y,z) in tri:
        if x == a and y == b:
            result.append((z,p3(x,y,z),"Trigram"))

    # If trigram unavailable, use bigram
    if not result:
        for (x,y) in bi:
            if x == b:
```

```

        result.append((y,p2(x,y),"Bigram"))

# If bigram unavailable, use unigram
if not result:
    result = [(x,p1(x),"Unigram") for x in uni]

return sorted(result,key=lambda x:x[1],reverse=True)[:5]

# Deleted interpolation
l1,l2,l3 = 0.2,0.3,0.5

def interpolation(sentence):

    w = sentence.split()
    a,b = w[-2],w[-1]
    result = []

    for word in uni:

        prob = (
            l1*p1(word)
            + l2*p2(b,word)
            + l3*p3(a,b,word)
        )

        result.append((word,prob))

    return sorted(result,key=lambda x:x[1],reverse=True)[:5]

sentence = "machine learning can"

print("BACKOFF:")
print(backoff(sentence))

print("\nDELETED INTERPOLATION:")
print(interpolation(sentence))

# Zero probability example
print("\nUnseen trigram probability:",
      p3("machine","learning","banana"))

```


Output:

```
... BACKOFF:
[('analyze', 0.3333333333333333, 'Trigram'), ('make', 0.3333333333333333, 'Trigram'), ('process', 0.3333333333333333, 'Trigram')]

DELETED INTERPOLATION:
[('analyze', 0.2212121212121212), ('make', 0.2212121212121212), ('process', 0.2212121212121212), ('solve', 0.05454545454545454), ('understand', 0.05454545454545454)]

Unseen trigram probability: 0.0
```

Q3. Entropy-Based Language Model Evaluation:

Code:

```
# Q3: ENTROPY-BASED LANGUAGE MODEL
```

```
from collections import Counter
import math
import re
```

```
train = """
the cat sits on the mat
the cat eats food
the dog sits on the mat
the dog eats food
the cat likes milk
machine learning is useful
machine learning uses data
artificial intelligence is powerful
language models predict words
"""
```

```
test = [
    "the cat sits on the mat",
    "the dog eats food",
    "the quantum processor redesigned the system"
]
```

```
words = re.findall(r'\b\w+\b',train.lower())
```

```
uni = Counter(words)
bi = Counter(zip(words[:-1],words[1:]))
tri = Counter(zip(words[:-2],words[1:-1],words[2:]))
```

```
V = len(uni)
```

```
# Probability functions
```

```
def p1(w):
    return uni[w]/len(words)
```

```
def p2(a,b):
    return bi[(a,b)]/uni[a] if uni[a] else 0
```

```
def p3(a,b,c):
    return tri[(a,b,c)]/bi[(a,b)] if bi[(a,b)] else 0
```

Entropy

```
def entropy(sentence,n):
```

```
    w = sentence.lower().split()
    total = 0
    count = 0
```

```
    for i in range(len(w)):
```

```
        if n == 1:
            p = p1(w[i])
```

```
        elif n == 2 and i > 0:
            p = p2(w[i-1],w[i])
```

```
        elif n == 3 and i > 1:
            p = p3(w[i-2],w[i-1],w[i])
```

```
        else:
            continue
```

```
        if p > 0:
            total += -math.log2(p)
            count += 1
```

```
    return total/count if count else float('inf')
```

Smoothed bigram entropy

```
def smooth_entropy(sentence):
```

```
    w = sentence.lower().split()
    total = 0
```

```
    for i in range(1,len(w)):
```

```
        p = (bi[(w[i-1],w[i])] + 1) / (uni[w[i-1]] + V)
```

```
        total += -math.log2(p)
```

```
    return total/(len(w)-1)
```

Test

```
for sentence in test:
```

```
    h1 = entropy(sentence,1)
    h2 = entropy(sentence,2)
```

```

h3 = entropy(sentence,3)
hs = smooth_entropy(sentence)

print("\nSentence:",sentence)

print("Unigram Entropy:",round(h1,3))
print("Bigram Entropy:",round(h2,3))
print("Trigram Entropy:",round(h3,3))
print("Smoothed Entropy:",round(hs,3))

if h2 < 3:
    print("Low predictive uncertainty")
else:
    print("High predictive uncertainty")

```

Output:

```

...
Sentence: the cat sits on the mat
Unigram Entropy: 3.622
Bigram Entropy: 0.923
Trigram Entropy: 0.396
Smoothed Entropy: 3.209
Low predictive uncertainty

Sentence: the dog eats food
Unigram Entropy: 3.87
Bigram Entropy: 0.936
Trigram Entropy: 0.5
Smoothed Entropy: 3.342
Low predictive uncertainty

```

Q4. Comparative POS Tagging :

Code:

Q4: COMPARATIVE POS TAGGING

```

import nltk
nltk.download('brown')
nltk.download('universal_tagset')

from nltk.corpus import brown
from collections import Counter,defaultdict

# Training corpus
data = brown.tagged_sents(
    tagset='universal'
)[:3000]

# Word -> tag frequency
word_tags = defaultdict(Counter)

```

```
# Tag transition frequency
transitions = Counter()
```

```
for sent in data:
```

```
    previous = "<START>"
```

```
    for word,tag in sent:
```

```
        word = word.lower()
```

```
        word_tags[word][tag] += 1
```

```
        transitions[(previous,tag)] += 1
```

```
        previous = tag
```

```
# Most frequent tag
```

```
most_tag = {
    w:c.most_common(1)[0][0]
    for w,c in word_tags.items()
}
```

```
# -----
```

```
# 1. Rule-Based Tagger
```

```
# -----
```

```
def rule_based(sentence):
```

```
    result=[]
```

```
    for word in sentence.split():
```

```
        w=word.lower()
```

```
        if w in ["i","you","he","she","we","they"]:
            tag="PRON"
```

```
        elif w in ["a","an","the"]:
            tag="DET"
```

```
        elif w.endswith("ly"):
            tag="ADV"
```

```
        elif w.endswith(("ing","ed")):
            tag="VERB"
```

```
        elif w.endswith(("ous","ful","ive")):
            tag="ADJ"
```

```
        elif w in ["in","on","at","with","for"]:
```

```
tag="ADP"
```

```
elif w in most_tag:  
    tag=most_tag[w]
```

```
else:  
    tag="NOUN"
```

```
result.append((word,tag))
```

```
return result
```

```
# -----  
# 2. Stochastic Tagger  
# -----
```

```
def stochastic(sentence):
```

```
    result=[]  
    previous("<START>")
```

```
    for word in sentence.split():
```

```
        w=word.lower()
```

```
        if w not in word_tags:
```

```
            tag=rule_based(word)[0][1]
```

```
        else:
```

```
            tags=word_tags[w]
```

```
            tag=max(  
                tags,  
                key=lambda t:  
                    tags[t] + transitions[(previous,t)]  
            )
```

```
            result.append((word,tag))  
            previous=tag
```

```
    return result
```

```
# -----  
# 3. Transformation-Based  
# -----
```

```
def transformation(sentence):
```

```
    result=rule_based(sentence)
```

```
for i,(word,tag) in enumerate(result):

    w=word.lower()

    # book as noun after determiner
    if w=="book":

        if i>0 and result[i-1][1]=="DET":
            result[i]=(word,"NOUN")
        else:
            result[i]=(word,"VERB")

    # read after pronoun is verb
    if w=="read" and i>0:
        if result[i-1][1]=="PRON":
            result[i]=(word,"VERB")

return result
```

```
# -----
# Test
# -----
```

```
sentences = [
    "I book a ticket",
    "I read a book"
]
```

```
for s in sentences:
```

```
    print("\nSentence:",s)
```

```
    print("\nRule-Based:")
    print(rule_based(s))
```

```
    print("\nStochastic:")
    print(stochastic(s))
```

```
    print("\nTransformation-Based:")
    print(transformation(s))
```

Output:

Sentence: I book a ticket

Rule-Based:

[('I', 'PRON'), ('book', 'NOUN'), ('a', 'DET'), ('ticket', 'NOUN')]

Stochastic:

[('I', 'NOUN'), ('book', 'NOUN'), ('a', 'NOUN'), ('ticket', 'NOUN')]

Transformation-Based:

[('I', 'PRON'), ('book', 'VERB'), ('a', 'DET'), ('ticket', 'NOUN')]

Sentence: I read a book

Rule-Based:

[('I', 'PRON'), ('read', 'VERB'), ('a', 'DET'), ('book', 'NOUN')]

Stochastic:

[('I', 'NOUN'), ('read', 'NOUN'), ('a', 'NOUN'), ('book', 'NOUN')]

Transformation-Based:

[('I', 'PRON'), ('read', 'VERB'), ('a', 'DET'), ('book', 'NOUN')]